

# Algoritmos e Estrutura de Dados III



Prova 1

Dia 11/03

## Complexidade de algoritmos

$\Omega$ : Limite inferior (crescimento mínimo).

$O$ : Limite superior (crescimento máximo).

$o$ : Limite superior estrito (crescimento mais rápido que o limite superior).

$\omega$ : Limite inferior estrito (crescimento mais lento que o limite inferior).

$\Theta$ : limite superior e inferior ao mesmo tempo

## Soma e Produto envolvendo complexidade

**Exemplo:**  $2n^2 + 100n + 56 = O(n^2) \Omega(n) o(n^3) \omega(\log n)$

Seja  $f_1(n) = O(g_1(n))$ ,  $f_2(n) = O(g_2(n))$ ,  $f_3(n) = O(g_3(n))$ .

- $f_1(n)f_2(n) = O(g_1(n)g_2(n))$
- $f(n)O(g(n)) = O(f(n)g(n))$
- $f_1(n) + f_2(n) = O(|g_1(n)| + |g_2(n)|)$
- $f_1(n) + f_3(n) = O(g_1(n))$
- $f(n) + O(g(n)) = O(f(n) + g(n))$
- $O(Kg(n)) = O(g(n))$
- Se  $f(n) = O(g(n))$ , então  $g(n) = \Omega(f(n))$
- Se  $f(n) = o(g(n))$ , então  $g(n) = \omega(f(n))$
- Se  $f(n) = O(g(n))$  e  $f(n) = \Omega(g(n))$ , então  $f(n) = \Theta(g(n))$

## Classe P, NP, NP-Difícil, NP-Completo e #P

A **classe P** contém todos os problemas que podem ser resolvidos em tempo polinomial (por exemplo,  $n^2$ ,  $n^3$ ,  $n^4$ , etc...) por uma máquina de Turing determinística. Ou seja, o problema está em P se existe um algoritmo que pode resolvê-lo com um número de operações que cresce de forma polinomial em relação ao tamanho de entrada.

A **classe NP** é composta por problemas de decisão cujas soluções podem ser verificadas em tempo polinomial. Não se sabe se todos esses problemas também podem ser resolvidos em tempo polinomial. Sabe-se que todo problema pertencente à classe **P** está contido em **NP**, porém permanece em aberto a questão de saber se  $P = NP$ .

Um problema **NP-Difícil** é aquele que é pelo menos tão difícil quanto qualquer outro problema em NP. Ou seja, se conseguíssemos resolver um problema NP-Difícil de forma eficiente, poderíamos também resolver todos os problemas da classe NP de forma eficiente.

Um problema **NP-Completo** é aquele que está na classe NP (pode verificar soluções em tempo polinomial) e é tão difícil quanto qualquer problema em NP.

A **classe #P** é uma extensão de NP, mas em vez de se preocupar com decidir se uma solução existe ou não, ela se preocupa em contar o número de soluções possíveis de um problema.

## Algoritmo determinístico VS não determinístico

**Algoritmo determinístico:** Um algoritmo determinístico é projetado para encontrar soluções precisas e bem definidas. Ele sempre produzirá a mesma saída para a mesma entrada, garantindo que a solução seja exata e correta, baseado na análise completa do problema. Exemplos típicos incluem algoritmos de ordenação como o Merge Sort ou a Busca Binária.

**Algoritmo não determinístico:** Um algoritmo não determinístico, ao contrário, não segue um único caminho fixo de execução. Ele pode explorar múltiplas possibilidades de forma paralela ou aleatória, utilizando abordagens como heurísticas (aproximação) ou probabilidade para encontrar soluções. Embora ele não forneça garantias de que a solução será ótima, ele é frequentemente mais rápido para encontrar soluções boas, especialmente em problemas complexos ou NP-difíceis, onde uma solução exata seria muito custosa. Um exemplo clássico são os algoritmos de busca local ou algoritmos probabilísticos.

Maneiras de mostrar que um problema **pertence a NP**:

- Mostrar que o problema pode ser resolvido por uma máquina de Turing não determinística em tempo polinomial
- Mostrar que, dado uma solução, é possível verificá-la em tempo polinomial com uma máquina de Turing determinística.

Passos para mostrar que um problema é **NP-Completo**:

- Mostrar que o problema pertence à classe NP, ou seja, dada uma solução, é possível verificá-la em tempo polinomial
- Mostrar que o problema é NP-Completo. É preciso pegar um problema NP-Completo conhecido e realizar uma redução polinomial dele para o problema analisado, mostrando que qualquer solução eficiente para esse caso também resolveria o problema NP-Completo.

## Teorema mestre

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

- $T(n)$ : Tempo de execução para resolver um problema de tamanho  $n$ .
- $a$ : Número de subproblemas em que o problema original é dividido.
- $n/b$ : Tamanho de cada subproblema.
- $f(n)$ : Custo adicional para dividir e combinar as soluções dos subproblemas (o trabalho não recursivo).

1. Caso 1:  $f(n) = O(n^{\log_b a})$ , tempo de execução é  $\Theta(n^{\log_b a})$
2. Caso 2:  $f(n) = \Theta(n^{\log_b a})$ , tempo de execução é  $\Theta(n^{\log_b a} \log n)$
3. Caso 3:  $f(n) = \Omega(n^{\log_b a})$ , tempo de execução é  $\Theta(f(n))$

Exemplos na parte II.

<https://lucaswithboots.github.io/unifal-cc-acervo/docs/disciplinas/aeds-3/2025/1/prova1.pdf>

<https://lucaswithboots.github.io/unifal-cc-acervo/docs/disciplinas/aeds-3/2024/1/prova1.pdf>

<https://lucaswithboots.github.io/unifal-cc-acervo/docs/disciplinas/aeds-3/2023/2/prova1.pdf>